

Отчёт по лабораторной работе №21

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Степанова Анастасия Михайловна № по списку 21.

Контакты: Nastya.stepanova1104@yandex.ru

Работа выполнена: «05» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «31» марта 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Динамические структуры данных. Обработка деревьев.

2. **Цель работы:** Составить программу на языке Си для построения и обработки упорядоченного двоичного дерева, содержащего узлы типа `int`. Реализовать основные функции работы с деревом (добавление, удаление, визуализация) и вычислить функцию в соответствии с вариантом.

3. **Задание (Вариант № 21):** Определить глубину минимальной вершины двоичного дерева.

4. **Оборудование:** Оборудование ПЭВМ студента, если использовалось:

Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ

5. **Программное обеспечение:** Программное обеспечение ЭВМ студента, если использовалось:

Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд `bash` версия 5.2.21.

Система программирования: `gcc` (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.

Язык программирования: Си

6. **Идея, метод, алгоритм решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:**

Идея: В упорядоченном двоичном дереве минимальная вершина — это узел с наименьшим значением ключа. Согласно свойству, минимальный элемент всегда находится в крайнем левом узле дерева.

Следовательно, глубина минимальной вершины — это количество рёбер от корня до этого крайнего левого узла. При этом для пустого дерева глубина не определена.

Метод:

1. **Построение дерева:** Дерево строится динамически. Каждый узел содержит целочисленный ключ (`data`), счётчик повторений (`cnt`), а также указатели на левое и правое поддеревья. Функция `insert` обеспечивает добавление нового узла с сохранением свойства упорядоченности.
2. **Поиск минимальной вершины:** Для нахождения минимальной вершины реализована функция `findMinDepth`. Она принимает корень дерева и последовательно переходит по указателям `left`, пока не достигнет узла, у которого левый потомок отсутствует. Этот узел и является минимальным.
3. **Вычисление глубины:** Функция `getMinDepth` использует алгоритм поиска минимальной вершины, но вместо возврата указателя на узел, она подсчитывает количество переходов (шагов) от корня до минимального узла, что и является глубиной.
4. **Обработка пустого дерева:** Предусмотрена проверка на пустое дерево. Если корень равен `NULL`, функция возвращает `-1`, что сигнализирует вызывающему коду об отсутствии результата.
5. **Алгоритм:**
Начало.
Если `root == NULL`:

Вернуть -1 (дерево пусто).
Инициализировать current = root, depth = 0.
Цикл, пока current->left != NULL:
current = current->left
depth = depth + 1
Конец цикла.
Вернуть depth.
Конец.

7. Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).

План работы:

1. Изучение теории: структура узла дерева, свойства упорядоченного двоичного дерева, методы обхода и удаления узлов.
2. Проектирование структуры данных: описать структуру Node с полями data, cnt, left, right.
3. Реализация базовых функций:
 - o createNode — создание нового узла.
 - o insert — вставка узла с учётом счётчика повторений.
 - o printTree — текстовая визуализация дерева (рекурсивный обход справа налево с отступами).
 - o deleteNode — удаление узла с учётом счётчика и перестроением дерева (поиск минимального узла в правом поддереве).
 - o freeTree — рекурсивное освобождение памяти.
4. Реализация функции для варианта 21: getMinDepth для вычисления глубины минимальной вершины.
5. Создание текстового меню для взаимодействия с пользователем.
6. Тестирование всех функций на различных наборах данных.

Тесты:

МЕНЮ

- 1 - Добавить число
- 2 - Показать дерево
- 3 - Удалить число
- 4 - Глубина минимальной вершины
- 5 - Завершить

Выполнить функцию:

1

Введите число: 50

МЕНЮ

...

4

Глубина минимальной вершины: 0

Соображения по тестированию:

Необходимо проверить корректность вставки узлов с построением различных конфигураций дерева (левая ветка, правая ветка, сбалансированное дерево), а также обработку дубликатов с увеличением счётчика. Важно протестировать удаление узлов в различных ситуациях: удаление листа, удаление узла с одним потомком, удаление узла с двумя потомками, удаление узла с несколькими вхождениями (счётчик > 1) и удаление несуществующего узла. Особое внимание следует уделить вычислению глубины минимальной вершины для пустого дерева (возврат -1), дерева из одного узла (глубина 0) и дерева с несколькими уровнями. Также необходимо проверить визуализацию дерева на корректность отображения уровней вложенности и устойчивость программы к некорректному вводу данных в меню.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <locale.h>
```

```
#include <windows.h>
```

```
typedef struct Node {
```

```
int data, cnt;

struct Node *left;

struct Node *right;

} Node;
```

```
Node* createNode(int value) {

    Node* newNode = (Node*)malloc(sizeof(Node));

    if (newNode == NULL)

        return NULL;

    newNode->data = value;

    newNode->cnt = 1;

    newNode->left = NULL;

    newNode->right = NULL;

    return newNode;

}
```

```
Node* insert(Node* root, int value) {

    if (root == NULL)

        return createNode(value);

    if (value < root->data) {

        root->left = insert(root->left, value);

    } else if (value > root->data) {

        root->right = insert(root->right, value);

    }

    else {

        root->cnt++;

    }

    return root;

}
```

```
Node* findMinDepth(Node* node) {

    Node* current = node;

    while (current && current->left != NULL) {

        current = current->left;

    }

    return current;

}
```

```
int findVal(Node* root, int value) {
```

```

if (root == NULL)
    return 0;
if (root->data == value)
    return 1;
if (value < root->data)
    return findVal(root->left, value);
else
    return findVal(root->right, value);
}

```

```

Node* deleteNode(Node* root, int value) {
    if (root == NULL)
        return root;
    if (value < root->data) {
        root->left = deleteNode(root->left, value);
    } else if (value > root->data) {
        root->right = deleteNode(root->right, value);
    } else {
        if (root->cnt > 1) {
            root->cnt--;
            return root;
        }
        if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }
        Node* temp = findMinDepth(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

```

```

void printTree(Node* root, int level) {

```

```

    if (root == NULL)
        return;
    printTree(root->right, level + 1);
    for (int i = 0; i < level; i++) printf("----");
    printf("%d (%d)\n", root->data, root->cnt);
    printTree(root->left, level + 1);
}

```

```

int getMinDepth(Node* root) {
    if (root == NULL)
        return -1;
    int depth = 0;
    Node* current = root;
    while (current->left != NULL) {
        current = current->left;
        depth++;
    }
    return depth;
}

```

```

void freeTree(Node* root) {
    if (root == NULL)
        return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

```

```

int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    setlocale(LC_ALL, "ru_RU.UTF-8");
    Node* root = NULL;
    int choice, val;
    while (1) {
        printf("\nМЕНЮ\n");
        printf("1 - Добавить число\n");
        printf("2 - Показать дерево\n");
        printf("3 - Удалить число\n");
    }
}

```

```
printf("4 - Глубина минимальной вершины\n");
printf("5 - Завершить\n");
printf("Выполнить функцию:\n");
if (scanf("%d", &choice) != 1) {
    printf("Ошибка ввода!\n");
    break;
}
switch (choice) {
    case 1:
        printf("Введите число: ");
        scanf("%d", &val);
        root = insert(root, val);
        break;
    case 2:
        if (root == NULL) {
            printf("Дерево пусто!\n");
            break;
        }
        printTree(root, 0);
        break;
    case 3:
        if (root == NULL) {
            printf("Дерево пусто, удалять нечего!\n");
            break;
        }
        printf("Какое число удалить:\n");
        scanf("%d", &val);
        if (!findVal(root, val)) {
            printf("Такого значения в дереве нет!\n");
        } else {
            root = deleteNode(root, val);
            printf("Число удалено\n");
        }
        break;
    case 4:
        val = getMinDepth(root);
        if (val == -1) {
            printf("Дерево пусто!\n");
        } else {
```

```

        printf("Глубина минимальной вершины: %d\n", val);
    }

    break;

case 5:

    freeTree(root);

    printf("Программа завершена.\n");

    return 0;

default:

    printf("Ошибка! Введите число от 1 до 5\n");

}

}

return 0;

}

```

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе выполнения лабораторной работы была разработана программа на языке Си, реализующая упорядоченное двоичное дерево с целочисленными ключами. Были реализованы основные операции: добавление элемента с учётом повторений, удаление элемента с корректным перестроением дерева, текстовая визуализация, а также функция для вычисления глубины минимальной вершины (вариант 21). Работа позволила закрепить навыки работы с динамическими структурами данных, рекурсивными алгоритмами и управлением памятью в языке Си.

Подпись студента _____